

音乐中断实验

一、实验介绍

在计算机系统中，外部中断是指在程序执行过程中，由外部输入触发的一种机制，用于暂停当前程序的执行，转而执行其他指定任务。外部中断的存在主要是为了处理紧急事件，例如硬件设备的输入、定时器的到期等。树莓派也可实现类似的功能。**本次实验任务为通过中断机制在树莓派上实现对不同外接设备的及时响应。**

二、实验原理

1. 树莓派中断函数

`add_event_detect(channel, edge, callback=None, bouncetime = None)`

- Channel: 需要检测状态变化的 GPIO 引脚
- Edge: 指定要监测的边沿类型，可以是 GPIO.RISING, GPIO.FALLING, 或者 GPIO.BOTH
- Callback: 可选参数，指定状态变化时需要执行的回调函数
- Bouncetime: 可选参数，用于消除抖动效应，指定两次状态变化之间的最小间隔(单位为 ms)



There are 2 kind of interrupts:

- RISING: when the state goes from LOW to HIGH.
- FALLING: when the state goes from HIGH to LOW.

2. GPIO.wait_for_edge(channel, GPIO.RISING)

主要用于在检测到边缘前阻止程序运行(占用 CPU 资源少)

三、实验要点

(1) 建立电路

结合之前实验接线连接不同的外部设备，如按键，LED 灯，蜂鸣器等。

(2) 编程思路

以控制流水灯状态为例，调用 `callback` 函数监视程序运行并及时中断。

使用 `add_event_detect` 函数实现用按键控制 LED 停止闪烁的示例程序如下：

```
import RPi.GPIO as GPIO
import time

# GPIO引脚配置
LED_PINS = [17, 27, 22, 5] # 流水灯的LED引脚
BUTTON_PIN = 18 # 按键引脚

# 全局变量
running = False # 流水灯运行状态

# 初始化GPIO
GPIO.setmode(GPIO.BCM)
GPIO.setup(LED_PINS, GPIO.OUT) # 配置LED引脚为输出
GPIO.setup(BUTTON_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP) # 按键使用上拉电阻

# 定义按键中断回调函数
def button_callback(channel):
    global running
    running = not running # 切换运行状态
    print(f"Running state changed to: {'ON' if running else 'OFF'}")

# 设置按键中断
GPIO.add_event_detect(BUTTON_PIN, GPIO.FALLING, callback=button_callback, bouncetime=200)

print("Press the button to control the LED flow. Press Ctrl+C to exit.")

try:
    while True:
        if running:
            # 流水灯逻辑
            for pin in LED_PINS:
                GPIO.output(pin, GPIO.HIGH) # 点亮当前LED
                time.sleep(0.2) # 延迟
                GPIO.output(pin, GPIO.LOW) # 熄灭当前LED
            else:
                time.sleep(0.1) # 按键暂停时降低CPU占用率
except KeyboardInterrupt:
    print("Exiting program.")
finally:
    GPIO.cleanup() # 清理GPIO设置
```

(3) 实验任务

完成音乐播放中断功能：在音乐正常播放时按下按键，音乐播放中断，LED 灯亮起，待 LED 灯闪烁几秒后自动继续播放之前的音乐。